Earthquake Detection and Alert System

This project is an IoT-based Earthquake Detection System designed using Arduino and ESP32 to monitor seismic activity in real time. The system uses an accelerometer sensor connected to the Arduino/ESP32 to detect ground vibrations and measure acceleration levels. When unusual seismic activity is detected, the ESP32 sends the data to a cloud-based infrastructure using HTTP requests.

The backend is built using AWS services, where API Gateway acts as the entry point to receive incoming sensor data. The data is then processed by an AWS Lambda function, which analyses the acceleration values and determines the severity of the earthquake based on predefined thresholds

.

Once a potential earthquake is detected, the system triggers AWS SNS (Simple Notification Service) to send real-time alerts to users in a specific region. These alerts include safety instructions and precautionary measures, helping users take immediate action to minimize risk.

This system provides a scalable, low-cost, and efficient solution for early earthquake warning, combining embedded systems with cloud computing to enhance public safety and disaster preparedness.

Arduino Uno

Arduino Uno acts as the main controller, which reads motion

data from an ADXL345 Accelerometer using I2C communication.

The accelerometer measures movement along the X, Y, and Z

axes and sends this data to the Arduino for processing.

Accelerometer

WiFi module like the ESP8266 WiFi Module is connected to the Arduino through serial communication, enabling internet connectivity. The Arduino collects the sensor data and transmits it to the WiFi module, which then sends the information to an online server or mobile application

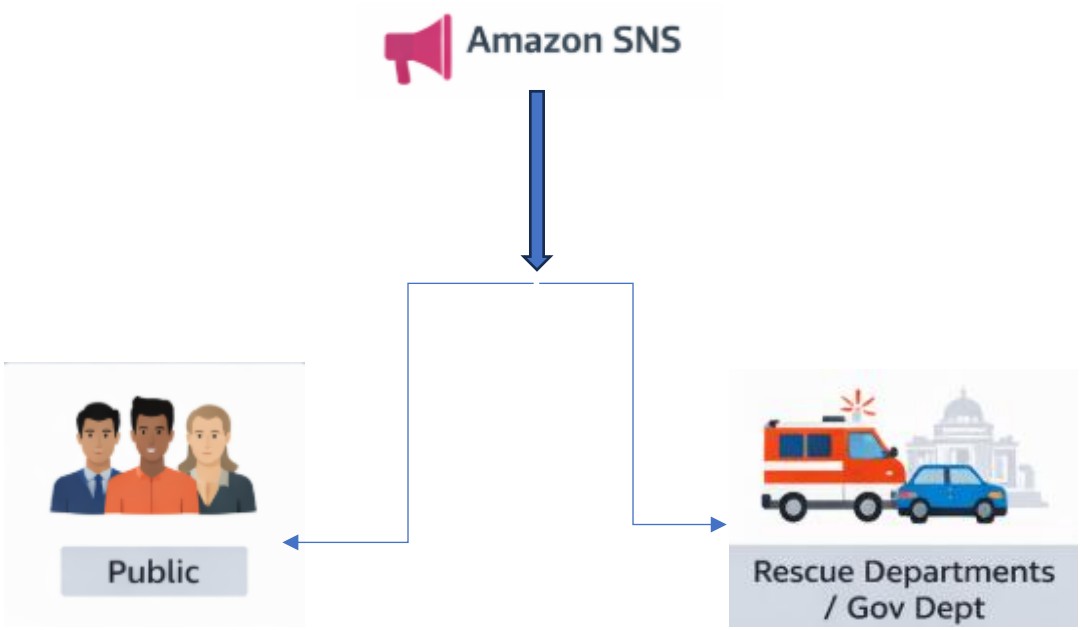
ESP8266

The Arduino Uno collects data from sensors and transmits it to the internet through a WiFi module such as the ESP8266 WiFi Module. Using this connection, the Arduino sends HTTP requests containing sensor data to Amazon API Gateway, which acts as an interface between the hardware device and cloud services.

```cpp
#include <WiFi.h>

const char* ssid = "hotspot_010";
const char* password = "Root@02";

void setup() {
  Serial.begin(115200);

  WiFi.begin(ssid, password);

  Serial.print("Connecting to WiFi");

  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.print(".");
  }

  Serial.println("\nConnected to WiFi!");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());
}
```

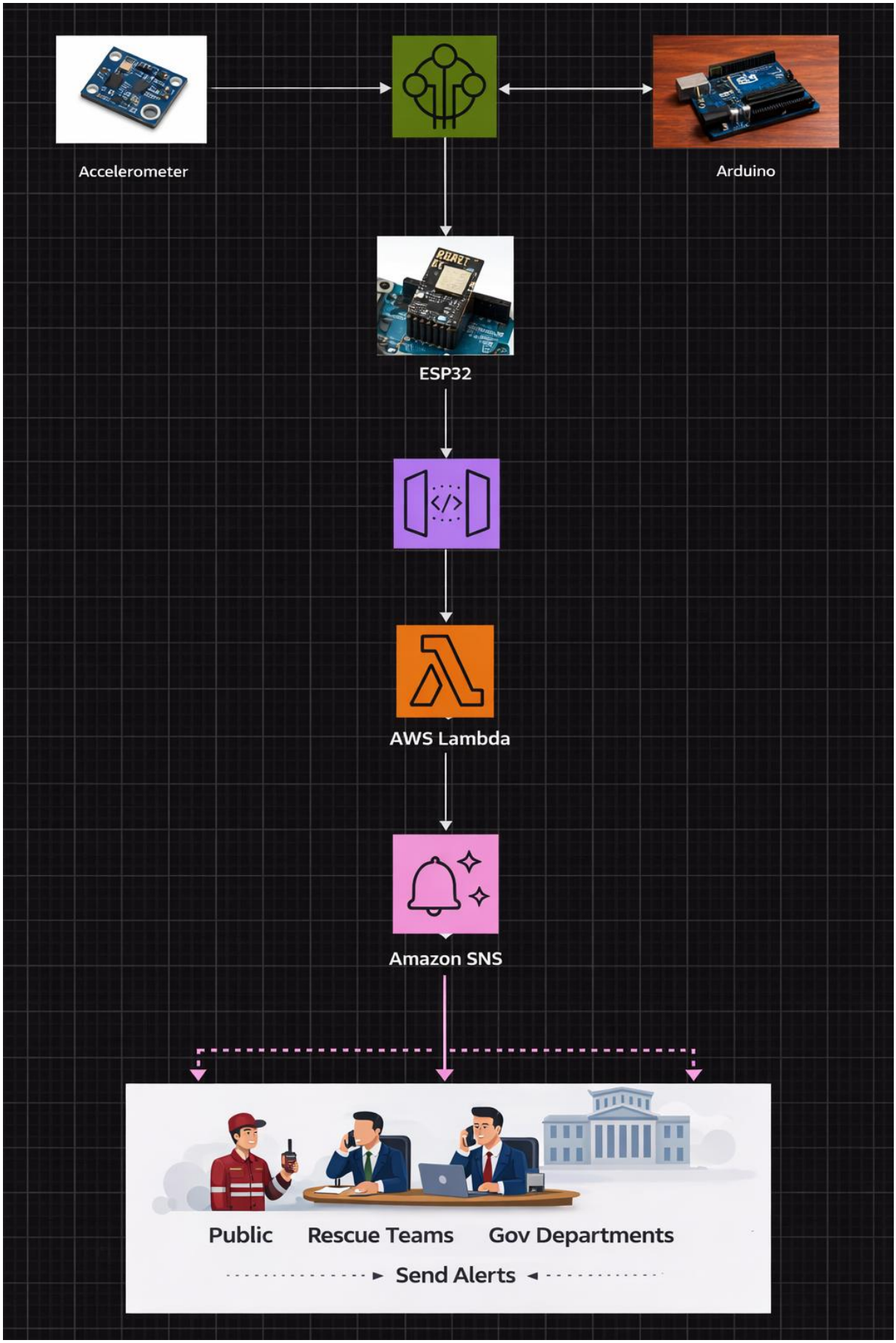
Amazon API Gateway receives the request and forwards it to AWS Lambda, a serverless computing service. Lambda processes the incoming data according to predefined logic, such as checking whether the sensor values exceed a certain threshold. If a specific condition is met, Lambda triggers a notification service

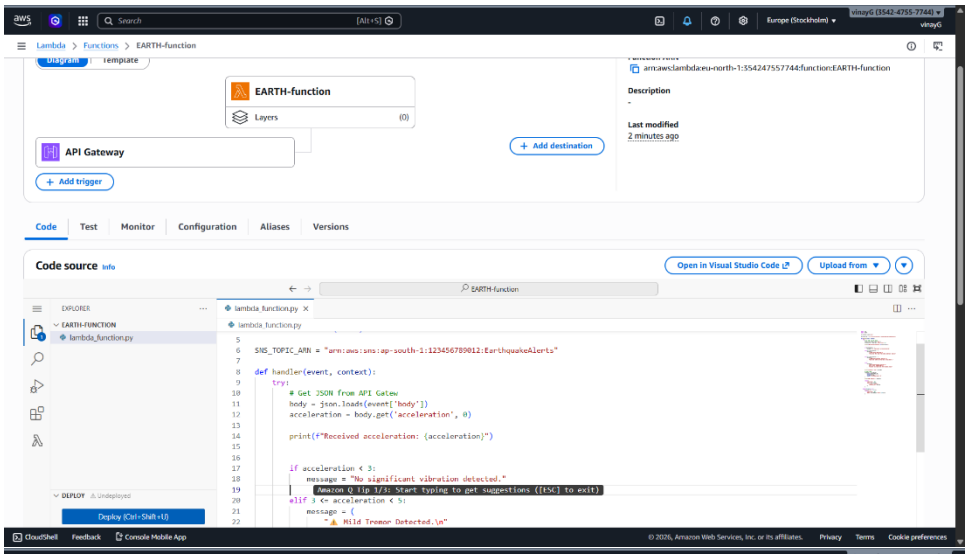
Amazon SNS

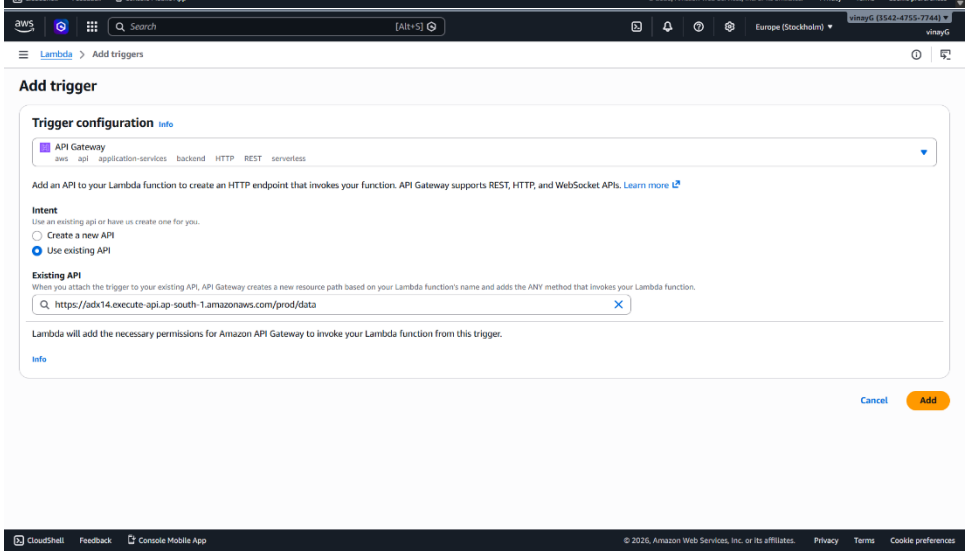


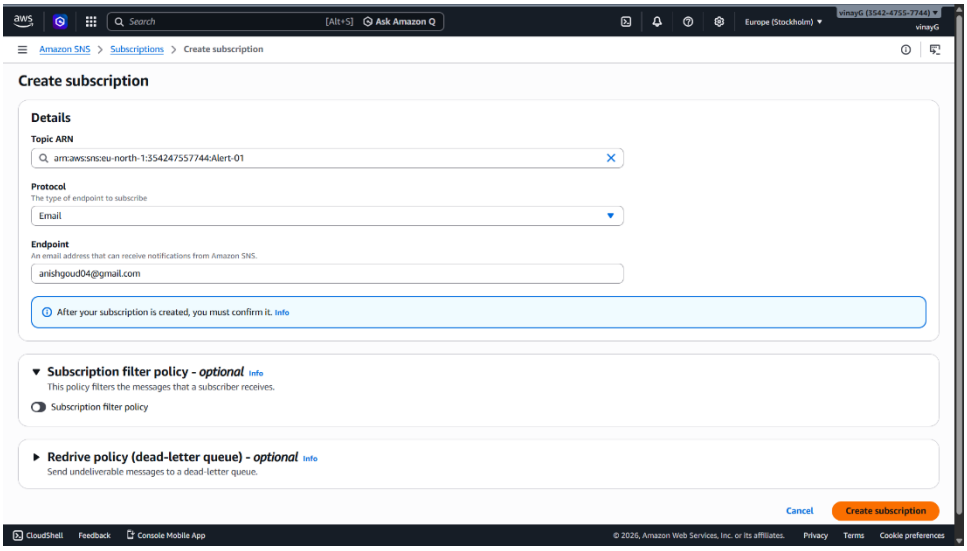
The notification is handled by Amazon SNS (Simple Notification Service), which is responsible for distributing messages. SNS sends SMS alerts to registered mobile numbers, informing users about important events detected by the system.

the integration of Arduino with AWS services enables real-time monitoring and instant communication, making the system suitable for applications like safety alerts, motion detection, and remote monitoring systems.


Accelerometer
Arduino
ESP32
AWS Lambda
Amazon SNS
Public
Rescue Teams
Gov Departments
Send Alerts


Lambda > Functions > EARTH-function
Diagram
Template
EARTH-function
Layers
(0)
API Gateway
Add trigger
Add destination
arn:aws:lambda:eu-north-1:354247557744:function:EARTH-function
Description
-
Last modified
2 minutes ago
Code
Test
Monitor
Configuration
Aliases
Versions
Code source Info
Open in Visual Studio Code
Upload from
EXPLORER
EARTH-FUNCTION
lambda_function.py
DEPLOY
Deploy (Ctrl+Shift+U)
SNS_TOPIC_ARN = "arn:aws:sns:ap-south-1:123456789012:EarthquakeAlerts"
def handler(event, context):
try:
# Get JSON from API Gatew
body = json.loads(event['body'])
acceleration = body.get('acceleration', 0)
print(f"Received acceleration: {acceleration}")
if acceleration < 3:
message = "No significant vibration detected."
Amazon Q tip 1/3: Start typing to get suggestions (tSC) to exit)
elif 3 <= acceleration < 5:
message = f
Mild Tremor Detected.\n"
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Lambda > Add triggers
Add trigger
Trigger configuration Info
API Gateway
aws
api
application-services
backend
HTTP
REST
serverless
Add an API to your Lambda function to create an HTTP endpoint that invokes your function. API Gateway supports REST, HTTP, and WebSocket APIs. Learn more
Intent
Use an existing api or have us create one for you.
Create a new API
Use existing API
Existing API
When you attach the trigger to your existing API, API Gateway creates a new resource path based on your Lambda function's name and adds the ANY method that invokes your Lambda function.
https://adx14.execute-api.ap-south-1.amazonaws.com/prod/data
Lambda will add the necessary permissions for Amazon API Gateway to invoke your Lambda function from this trigger.
Info
Cancel
Add
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Search
[Alt+S]
Ask Amazon Q
Europe (Stockholm)
vinayG (3542-4755-7744)
vinayG
Amazon SNS > Subscriptions > Create subscription
Create subscription
Details
Topic ARN
arn:aws:sns:eu-north-1:354247557744:Alert-01
Protocol
The type of endpoint to subscribe
Email
Endpoint
An email address that can receive notifications from Amazon SNS.
anishgoud04@gmail.com
After your subscription is created, you must confirm it. Info
Subscription filter policy - optional Info
This policy filters the messages that a subscriber receives.
Subscription filter policy
Redrive policy (dead-letter queue) - optional Info
Send undeliverable messages to a dead-letter queue.
Cancel
Create subscription
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <Wire.h>
#include <MPU6050.h>

MPU6050 mpu;

const char* ssid = "hotspot_010";
const char* password = "Root@02";
const char* serverName = "https://adx14.execute-api.ap-south-1.amazonaws.com/prod/data";

void setup() {
  Serial.begin(115200);
  Wire.begin();

  mpu.initialize();
```

```
  if (mpu.testConnection()) {
    Serial.println("MPU6050 Connected");
  } else {
    Serial.println("MPU6050 Connection Failed");
  }

  WiFi.begin(ssid, password);
  Serial.print("Connecting");

  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.print(".");
  }

  Serial.println("\nWiFi Connected");
}

void loop() {
  int16_t ax, ay, az;
```

```
float accel = sqrt(ax * ax + ay * ay + az * az) / 16384.0;

Serial.print("Acceleration: ");
Serial.println(accel);

if (accel >= 3.0 && accel <= 9.5) {
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;

    http.begin(serverName);
    http.addHeader("Content-Type", "application/json");

    String jsonData = "{\"acceleration\": " + String(accel) + "}";

    int httpResponseCode = http.POST(jsonData);

    Serial.print("HTTP Response code: ");
    Serial.println(httpResponseCode);
```

```
      http.end();
    }
  }

  delay(2000);
}
```